Aplicación Web MVC con Spring Boot y Thymeleaf

Gestión de Alumnos y Carreras: CRUD Completo, Relación @ManyToOne, DTOs y Validación Jakarta

Java 21 LTS Spring Boot 4.x MySQL 8 + Hibernate

Instructor: Dr. José Asunción Enríquez Zárate

ARQUITECTURA Y FUNDAMENTOS

Stack Tecnológico del Proyecto

Spring Boot & Web

Controladores MVC modernos, inyección de dependencias y productividad con Lombok.

Thymeleaf SSR

Plantillas HTML5 naturales renderizadas en servidor con diseño responsive Bootstrap 5.

Spring Data JPA

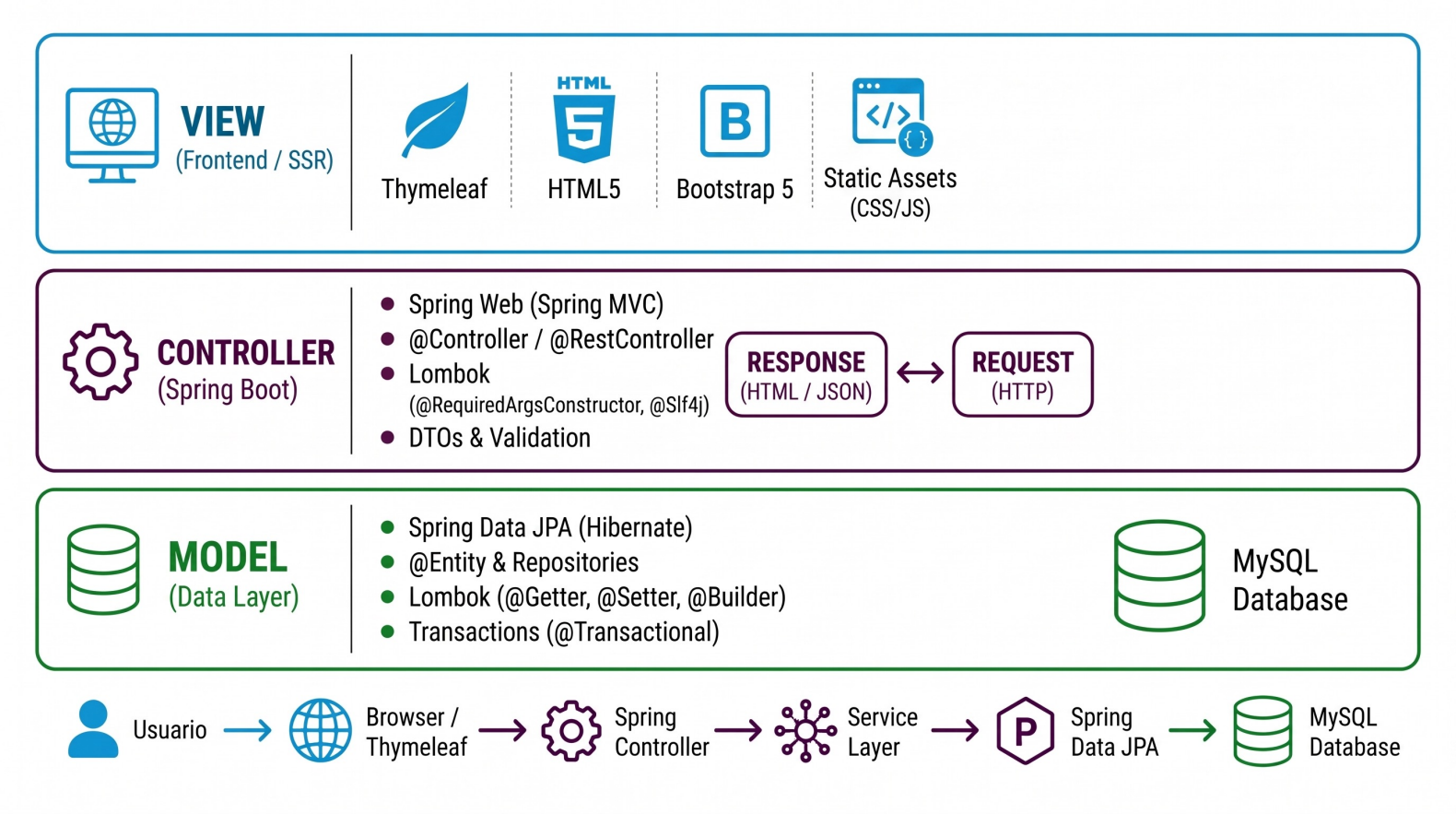
Persistencia relacional con Hibernate sobre motor MySQL 8 y gestión transaccional.

Jakarta Validation

Reglas de negocio y sanitización de formularios con BindingResult en capas desacopladas.

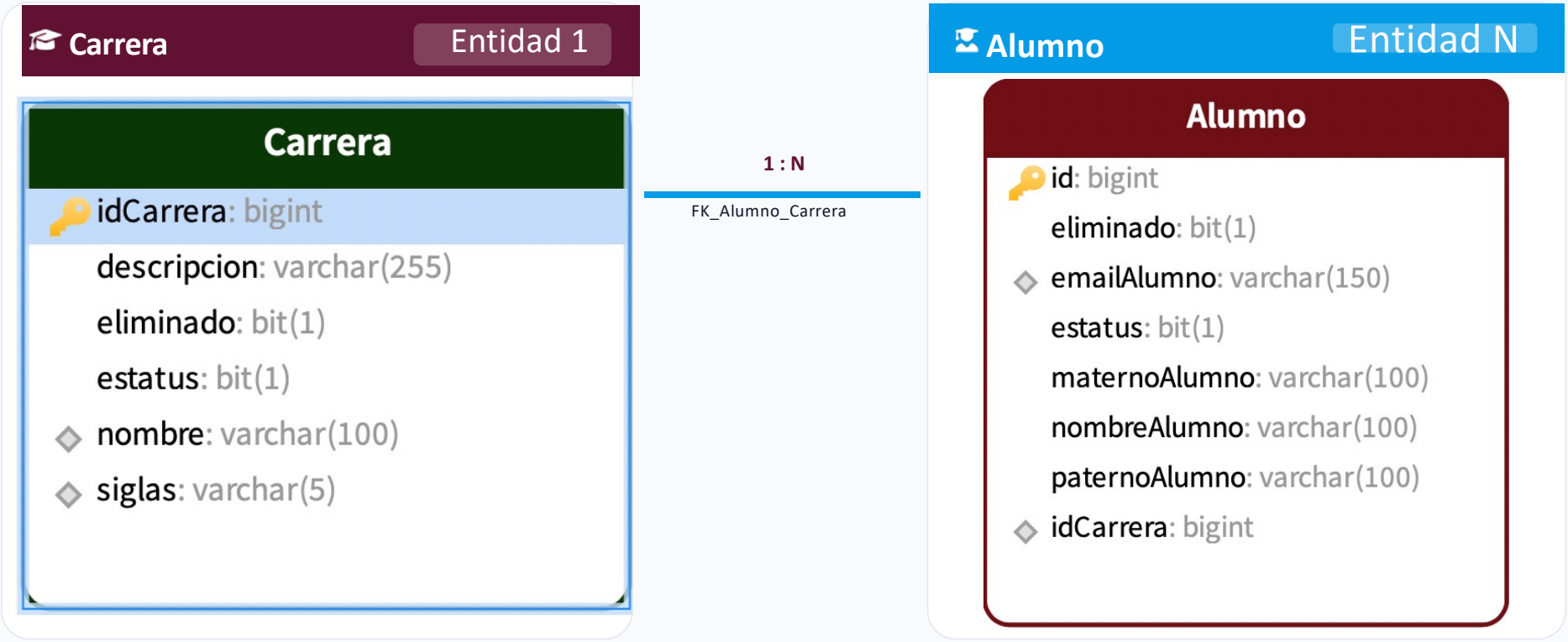
ARQUITECTURA Y FUNDAMENTOS

Stack Tecnológico del Proyecto



BLOQUE 0 · ESTRUCTURA RELACIONAL

Modelo Entidad-Relación: Carrera y Alumno



BLOQUE 0 · CREACIÓN DEL ESQUEMA

Script DDL Base de Datos MySQL

```
CREATE DATABASE IF NOT EXISTS TallerSpringMVC;
 USE TallerSpringMVC;

 CREATE TABLE Carrera (
   idCarrera BIGINT AUTO_INCREMENT,
   siglas VARCHAR(5) NOT NULL,
   nombre VARCHAR(100) NOT NULL,
   descripcion VARCHAR(255),
   estatus TINYINT(1) DEFAULT 1,
   eliminado TINYINT(1) DEFAULT 0,
   PRIMARY KEY (idCarrera),
   UNIQUE KEY (siglas),
   UNIQUE KEY (nombre)
 );
```

```
CREATE TABLE Alumno (
  idAlumno BIGINT AUTO_INCREMENT,
  nombreAlumno VARCHAR(100) NOT NULL,
  paternoAlumno VARCHAR(100) NOT NULL,
  maternoAlumno VARCHAR(100),
  emailAlumno VARCHAR(150) NOT NULL,
  estatus TINYINT(1) DEFAULT 1,
  eliminado TINYINT(1) DEFAULT 0,
  idCarrera BIGINT NOT NULL,
  PRIMARY KEY (idAlumno),
  UNIQUE KEY (emailAlumno),
  FOREIGN KEY (idCarrera)
    REFERENCES Carrera (idCarrera)
    ON DELETE RESTRICT ON UPDATE CASCADE
 );
```

BLOQUE 0 · RESULTADO ESPERADO

Módulos Funcionales del Sistema Académico

/TallerWebMVCSpring/

Dashboard Principal

Portal unificado con accesos rápidos a gestión académica y barra de navegación institucional modularizada.

/carreras

Catálogo de Carreras

Tabla dinámica con siglas, nombre, descripción, estado activo/inactivo y acciones de edición y borrado.

/alumnos

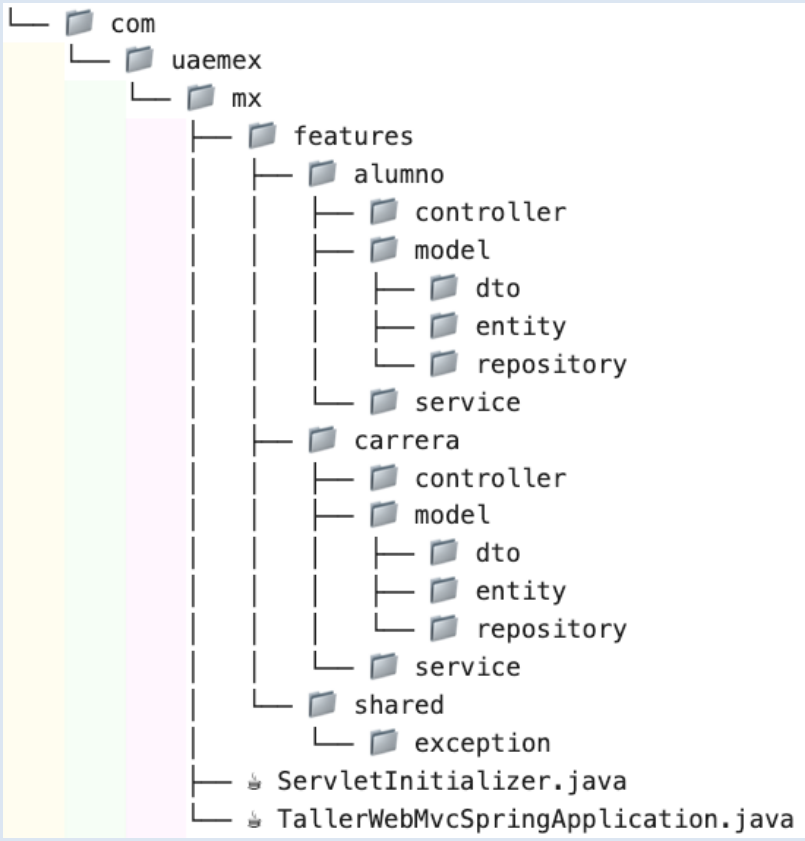
Matrícula de Alumnos

Listado relacional que muestra siglas y carrera vinculada mediante `@ManyToOne` y selector en formulario.

Dependencias y Estructura Feature-Based

Spring Initializr

- **Spring Web:** Controladores MVC
- **Thymeleaf:** Motor de plantillas SSR
- **Spring Data JPA:** Capa de persistencia
- **MySQL Driver:** Conector oficial
- **Validation & Lombok:** Productividad



BLOQUE 1 · DISEÑO DE SOFTWARE

Arquitectura MVC en Tres Niveles

1. VIEW (Frontend / SSR)
Thymeleaf · HTML5 · Bootstrap 5 · Recursos Estáticos (CSS/JS)

2. CONTROLLER (Spring Boot MVC)
@Controller · DTOs & Validation · @RequiredArgsConstructor · BindingResult

3. MODEL & DATA LAYER
Service Layer (@Transactional) · Spring Data JPA (Hibernate) · MySQL 8

Dr. José Asunción Enríquez Zárate

Configuración de application.properties

```
# Conexión MySQL
spring.datasource.url=jdbc:mysql://localhost:3306/TallerSpringMVC?useSSL=false&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=${DB_PASSWORD:laClaveAqui}
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true

spring.jpa.hibernate.naming.physical-strategy
=org.hibernate.boot.model.naming.PhysicalNamingStrategyStandardImpl
spring.jpa.hibernate.naming.implicit-strategy
=org.hibernate.boot.model.naming.ImplicitNamingStrategyLegacyJpaImpl


# Thymeleaf
spring.thymeleaf.cache=false
spring.thymeleaf.prefix=classpath:/templates/
spring.thymeleaf.suffix=.html
```

BLOQUE 3 · MODELO DE ENTIDADES

Entidad Primaria Carrera y Relación @ManyToOne

```
// Relación Foránea en Alumno.java
@ManyToOne(fetch = FetchType.LAZY, optional = false)
@JoinColumn(
  name = "idCarrera", nullable = false,
  foreignKey = @ForeignKey(name = "fk_alumno_carrera")
)
@ToString.Exclude
@EqualsAndHashCode.Exclude
private Carrera carrera;
```

FetchType.LAZY
Evita cargar datos innecesarios en cada consulta de Alumno.

Previene Recursión Infinita
evita bucles infinitos al imprimir entidades con Lombok.

@ToString.Exclude

Anotaciones Lombok
Uso de @Data, y @NoArgsConstructor @AllArgsConstructor

BLOQUE 3 · MODELO DE DTOS

Separación de DTO de Lectura vs Escritura

CarreraDTO / AlumnoDTO

DTO de Lectura / Salida

- Envía información desde el servidor hacia la vista o cliente.
- Incluye el identificador único (`idCarrera` / `idAlumno`).
- Representación segura y completa sin proxies de Hibernate.

CreateOrUpdateDTO

DTO de Entrada / Validación

- Recibe y valida datos desde formularios HTML.
- Contiene anotaciones Jakarta Validation (`@NotBlank`, `@Size`).
- Protege la integridad de las entidades JPA.

BLOQUE 3 · ACCESO A DATOS

Interfaces Repositories y Query Methods

CarreraRepository

```
boolean existsByNombre(String nombre);
boolean existsBySiglas(String siglas);
```

Consultas derivadas automáticas para evitar duplicidad de claves únicas antes de persistir.

AlumnoRepository

```
Optional findByEmailAlumno(String email);
boolean existsByEmailAlumno(String email);
boolean existsByCarreraIdCarrera(Long id);
```

Valida unicidad de correo electrónico y verifica si una carrera tiene alumnos asignados antes de borrarla.

BLOQUE 3 · LÓGICA DE NEGOCIO

Capa Service e Implementación Desacoplada

CarreraService / CarreraServiceImpl

+ listar() : List
+ guardar(CreateOrUpdateCarreraDTO)
+ buscarPorId(Long) : CarreraDTO
+ actualizar(Long, CreateOrUpdateDTO)
+ eliminar(Long) : void
- mapearADTO / mapearAEntity

AlumnoService / AlumnoServiceImpl

+ listar() : List
+ guardar(CreateOrUpdateAlumnoDTO)
+ buscarPorId(Long) : AlumnoDTO
+ actualizar(Long, CreateOrUpdateDTO)
+ eliminar(Long) : void
- mapearADTO(Alumno) : AlumnoDTO

BLOQUE 3 · CAPA CONTROLADOR

CarreraController y HomeController

CarreraController

- **Desacoplamiento:** Solo recibe CreateOrUpdateCarreraDTO, nunca entidades JPA.
- **Control de Errores:** Integra BindingResult para retornar el formulario con avisos.
- **Patrón PRG:** Usa RedirectAttributes para prevenir doble envío de formularios.

HomeController

- **Responsabilidad:** Método index() mapea la ruta raíz (/).
- **Separación (SoC):** Aísla la navegación general de la lógica CRUD de negocio.

BLOQUE 3 · CAPA CONTROLADOR

AlumnoController y @ExceptionHandler Local

Operaciones y BindingResult

- Métodos listar, nuevo, guardar, editar, actualizar y eliminar.
- Carga catálogo dinámico de carreras en los formularios de alta y edición.
- Valida DTO de captura mediante BindingResult.

Método Especial: aplicarError()

- Anotado con @ExceptionHandler(RuntimeException.class)
- Intercepta reglas de negocio (ej. email duplicado) dentro del controlador.
- Inyecta el error directamente en BindingResult para mostrarlo en el HTML.

BLOQUE 3 · TOLERANCIA A FALLOS

GlobalExceptionHandler (@ControllerAdvice)

Seguridad

Evita exponer trazas SQL, nombres de clases o rutas internas del servidor en pantalla.

Experiencia UX

Muestra una página de error personalizada con el branding institucional y mensaje amigable.

Mantenibilidad

Centraliza en handleGlobalError() el registro de logs y notificaciones.

BLOQUE 4 · RUTAS WEB Y LÓGICA

Endpoints CRUD: Alumnos y Carreras

Método	Ruta HTTP	Controlador	Acción
GET	/alumnos · /carreras	Alumno / Carrera	Lista todos los registros activos
GET	/alumnos/nuevo · /carreras/nuevo	Alumno / Carrera	Carga formulario vacío y catálogos
POST	/alumnos/guardar · /carreras/guardar	Alumno / Carrera	Valida con BindingResult y redirige
POST	/alumnos/editar/{id} · /carreras/editar/{id}	Alumno / Carrera	Actualiza registro por identificador
POST	/alumnos/eliminar/{id} · /carreras/eliminar/{id}	Alumno / Carrera	Borra registro por identificador

BLOQUE 6 · PRUEBAS EN NAVEGADOR

Flujo de Verificación y Persistencia

01

Crear Carreras

Registrar catálogo inicial (ej. LIA, ISC) para habilitar llaves foráneas.

02

Registrar Alumno

Asignar carrera mediante selector dinámico y validar guardado en BD.

03

Probar Validaciones

Provocar email inválido o campos vacíos para verificar mensajes inline.

04

Editar y Borrar

Actualizar datos, probar borrado y verificar integridad en MySQL Workbench.

BLOQUE 5 · MOTOR DE PLANTILLAS

Thymeleaf: Natural Templates y Usos

¿Qué es Natural Template?

Archivos HTML5 válidos que pueden abrirse directamente en el navegador sin levantar el servidor Spring Boot, facilitando el trabajo colaborativo entre diseño y backend.

```
th:text="${mensaje}">Vista previa
```

Capacidades Clave en Spring

- **Renderizado SSR:** Compila HTML limpio antes de responder HTTP.
- **Binding Bidireccional:** Enlaza DTOs y formularios HTML.
- **Seguridad XSS:** Escapa caracteres especiales por defecto.
- **Modularización:** Reutilización de layouts con fragmentos.

Expresiones Estándar (Standard Expression Syntax)

Símbolo	Tipo de Expresión	Propósito	Ejemplo
${...}	Variable Expressions	Evalúa atributos del Model de Spring	${usuario.nombre}
*{...}	Selection Expressions	Propiedades sobre objeto seleccionado (th:object)	*{apellido}
#{...}	Message (i18n)	Mensajes desde messages.properties	#{btn.guardar}
@{...}	Link URL Expressions	Rutas relativas al Context-Path con parámetros	@{/alumnos(id=${c.id})}
~{...}	Fragment Expressions	Carga fragmentos reutilizables	~{fragments/nav :: menu}

BLOQUE 5 · DIRECTIVAS BASE

Variables, Selección de Formularios y URLs

```
// 1. Variable & XSS
<h1 th:text="${titulo}"></h1>

// Sin escapar HTML:
<div th:utext="${html}"></div>
```

```
// 2. Selección th:object
<form th:action="@{/carreras/guardar}"
      th:object="${carrera}" method="post">
  <input th:field="*{nombre}" />
</form>
```

```
// 3. Enlaces @{...}
<a
th:href="@{/alumnos/{id}(id=${a.id})}">
  Ver Perfil
</a>
<img th:src="@{/img/logo.svg}" />
```

BLOQUE 5 · CONTROL DE FLUJO EN VISTAS

Iteración (th:each) y Condicionales (th:if / th:unless)

```
// Iteración de colecciones
<tr th:each="carrera : ${carreras}">
  <td th:text="${carrera.idCarrera}">1</td>
  <td th:text="${carrera.siglas}">ISC</td>
  <td th:text="${carrera.nombre}">Sistemas</td>
</tr>
```

```
// Renderizado condicional
<div th:if="${mensajeError}" class="alert">
  <span th:text="${mensajeError}"></span>
</div>

<div th:unless="${not #lists.isEmpty(carreras)}">
  Sin registros disponibles.
</div>
```

BLOQUE 5 · FORMULARIOS AVANZADOS

Validación Inline (#fields) y Campos Ocultos

```
// Retroalimentación visual Bootstrap 5
<input type="text”
  th:field="*{siglas}"
       th:class="${#fields.hasErrors('siglas')} ?
       'form-control is-invalid' : 'form-control'" />

<div class="invalid-feedback"
    th:if="${#fields.hasErrors('siglas')}"
    th:errors="*{siglas}">Error</div>
```

```
// Conservación de estado en UPDATE
<form th:action="@{/carreras/guardar}"
th:object="${carrera}">
  <!-- Diferencia INSERT de UPDATE -->
  <input type="hidden" th:field="*{idCarrera}" />
  <input type="hidden" th:field="*{eliminado}" />
</form>
```

BLOQUE 5 · MODULARIZACIÓN DRY

Fragments: th:fragment, th:replace y th:block

```
/ Definición (fragments/footer.html)
<footer th:fragment="footer">
  <p>&copy; 2026 Gestión Académica</p>
</footer>

// Inclusión en vistas
<div th:replace="~{fragments/footer :: footer}"></div>
<th:block th:insert="~{fragments/header :: head}">
</th:block>
```

th:insert vs th:replace
insert coloca el fragmento dentro de la etiqueta anfitriona; **replace** sustituye completamente el tag anfitrión.

th:block
Contenedor virtual que desaparece al compilarse, evitando divs residuales en el DOM.

BLOQUE 5 · ARQUITECTURA LIMPIA

Buenas Prácticas en Thymeleaf & Spring Boot

Caché por Entorno

Desactivar caché en desarrollo (cache=false) y activarla en producción.

DTOs en Vista

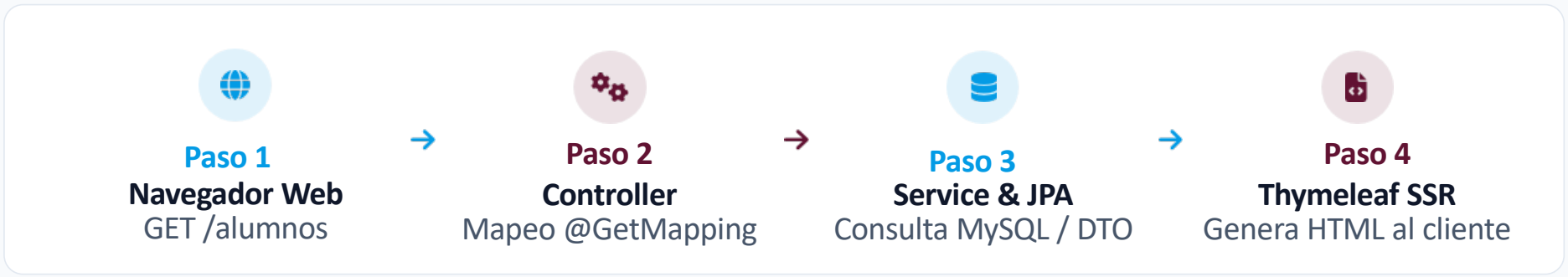
Favorecer DTOs específicos frente a entidades JPA para prevenir LazyInitializationException.

Sanitización y URLs

Priorizar `th:text` contra XSS y estandarizar rutas con `@{/...}`.

ARQUITECTURA EN CAPAS

Flujo de Petición MVC con Thymeleaf



Diferencia Clave vs API REST: El servidor compila la página HTML completa con los datos inyectados, eliminando la necesidad de hidratación en clientes JS complejos.

Manejo Transaccional: La capa Service coordina las entidades y su mapeo a DTOs de transporte para evitar fugas del modelo de dominio.

BLOQUE 4 · RUTAS WEB Y LÓGICA

Endpoints CRUD: Alumnos y Carreras

Método	Ruta HTTP	Controlador	Acción
GET	/alumnos · /carreras	Alumno / Carrera	Lista todos los registros activos
GET	/alumnos/nuevo · /carreras/nuevo	Alumno / Carrera	Carga formulario vacío y catálogos
POST	/alumnos/guardar · /carreras/guardar	Alumno / Carrera	Valida con BindingResult y redirige
POST	/alumnos/editar/{id} · /carreras/editar/{id}	Alumno / Carrera	Actualiza registro por identificador
POST	/alumnos/eliminar/{id} · /carreras/eliminar/{id}	Alumno / Carrera	Borra registro por identificador

GUÍA DEL INSTRUCTOR · CONTINGENCIA

Matriz de Riesgos y Estrategias de Mitigación

Riesgo Potencial	Estrategia de Mitigación
Lombok no compila getters/setters	Validar plugin del IDE previamente y tener de respaldo listo. pom.xml
MySQL no conecta o rechaza credenciales	Activar Plan B con base de datos H2 en memoria mediante datasource de testing.
Tiempo insuficiente para maquetar Thymeleaf	Contar con plantillas base precargadas en repositorio de apoyo para copiar.
Select de carrera vacío en formulario	Ejecutar el script SQL inicial con carreras precargadas antes del bloque práctico.

Dr. José Asunción Enríquez Zárate

EVALUACIÓN Y CIERRE

Checklist de Entregables del Taller

- [x] **Proyecto Spring Boot 4.x compilando y conectado a MySQL 8**
- [x] **Entidades Carrera y Alumno con relación @ManyToOne, Lombok y Validation**
- [x] **Repositorios JPA y capa de servicio desacoplada con DTOs**
- [x] **Vistas Thymeleaf con selector dinámico de carrera y feedback de errores**
- [x] **CRUD completo funcional y navegable desde la interfaz web**

Dr. José Asunción Enríquez Zárate

¿Preguntas o Dudas?

starter/ · final/ · script.sql

https://github.com/silentghostphantom/TallerSpringMVC

Impartido por **Dr. José Asunción Enríquez Zárate** · Taller de Desarrollo Web MVC